DESARROLLO DE APLICACIONES CON ANGULAR

Angular

ANGULAR

https://angular.io/guide/testing

ANGULAR

Testing

- Se crean en ficheros con la extensión spec.ts.
- Se ejecutan con **ng test**
- Los grupos de pruebas de agrupan en grupos o 'suites' delimitados por la función **describe** que recibe como parámetros un nombre y una función anónima.

```
describe('Nombre del grupo', () => {

});
```

- fdescribe → sólo se ejecuta esa suite de pruebas.
- xdescribe → No se ejecuta esa suite de pruebas.

ANGULAR

Testing

- Dentro de la función anónima:
  - Método **it**→Contiene la prueba

    ```
    it('descripción', ()=>{

    }
    ```

  - Método **it**→Contiene la evaluación de los resultados (método **expect**).

    ```
    it('descripción', ()=>{
      expect(elemento-a-evaluar).toXXX();
    });
    ```

    - Puede haber varios **expect** por cada **it**.
  - Puede haber varios **it** por cada **describe**.
  - Cada **it** es independiente y tiene su propio contexto.
  - **fit** si se quiere que solo se ejecute esa prueba.
  - **xit** si no se quiere que se ejecute esa prueba.

ANGULAR

- Testing
  - Método beforeEach
    - Permite agrupar funcionalidad común a los diferentes test.
    - Se ejecuta antes de cada método **it**.
    - beforeEach(()=>{

      }
  - Método afterEach
    - Se ejecuta después de cada método **it.**
    - afterEach(()=>{

      }

ANGULAR

- Testing
  - Objeto testBed
    - Permite configurar las pruebas como si fuesen un módulo.
    - Permite inyectar elementos (por ejemplo servicios) evitando instanciarlos directamente → "forma Angular".
    - En beforeEach:

```
TestBed.configureTestingModule({
    providers: [
        ClaseServicio
    ]
});
nombreServicio = TestBed.inject(ClaseServicio);
```

ANGULAR

- Testing
  - Módulo HttpTestingModule
    - Permite realizar pruebas http simulando las respuestas.
    - Dentro de **describe**, se declara un atributo de tipo HttpTestingController
      - **let *nombreController:*HttpTestingController;**
    - En beforeEach:

```
TestBed.configureTestingModule({
    imports: [ HttpClientTestingModule ],
    providers: [
        ClaseServicio
    ]
});
nombreServicio = TestBed.inject(ClaseServicio);
nombreController = TestBed.inject(HttpTestingController)
```

ANGULAR

- Testing
  - Módulo HttpTestingModule
    - Prueba:

```
it('descripción de la prueba', () =>
    nombreServicio.nombreMétodo().subscribe(respuesta => {
            expect(respuesta).toBeXXX…
    });
    const req=nombreController.expectOne('http://….');
    expect(req.request.method).toBe('GET');
    req.flush({
            //Datos de la simulación de la respuesta
    });
```

ANGULAR

- Testing
  - Módulo HttpTestingModule
    - Comprobación de peticiones únicas (no se duplican):
      - En afterEach:
        - *nombreController.*verify()

ANGULAR

- Testing
  - Pruebas sobre componentes:

```
beforeEach(waitForAsync(()=> {
    TestBed.configurateTestingModule({
            imports:[AppModule]
    }).compileComponents()
            .then(()=>{
                    componente = TestBed.createComponente(NombreComponente);
                    instancia = componente.componentInstance;
            })
 }));
it('Debe existir el componente',()=>{
    expect(instancia).toBeTruthy();
}
```

Se puede importar componentes individuales o módulos

ANGULAR

- Testing
  - Pruebas sobre componentes:
    - Referencias habituales:
      - Componente → El componente que se quiere probar
      - ComponenteFixture<*ClaseComponente*> → Accesorio para probar y depurar el compoennte. Es el proveedor de las instancias de los componentes y del DebugElement.
      - DebugElement →Elemento de depuración que da acceso al DOM.
    - En beforeEach, mediante TestBed se configura (importación e inicialización) de los componentes.

ANGULAR

- Testing. Pruebas por defecto.

```
describe('Componente21Component', () => {
  let component: Componente21Component;
  let fixture: ComponentFixture<Componente21Component>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      declarations: [ Componente21Component ]
    })
    .compileComponents();
  });

  beforeEach(() => {
    fixture = TestBed.createComponent(Componente21Component);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });
});
```

ANGULAR

- Testing. Ejemplo de prueba.

```
<div id="principal">Contenidox</div>

it('Debe existir el contenido', ()=>{
    el = fixture.debugElement;
    expect(el.query(By.css('#principal')).nativeElement.textContent).toBe("Contenido","Debe poner Contenido");
})
```

ANGULAR

- Testing
  - Pruebas sobre componentes:
    - Tipo DebugElement → Elementos de depuración.
      - let debEle:DebugElement;
      - Obtención de la referencia:

```
compileComponents()
.then(()=>{
        componente = TestBed.createComponente(NombreComponente);
        instancia = componente.componentInstance;
        debEle = componente.debugElement;
})
```

ANGULAR

- Testing
  - Pruebas sobre componentes:
    - Tipo DebugElement → Elementos de depuración.
      - let debEle:DebugElement;
      - Sobre la referencia debEle podemos utilizar selectores para acceder a los elementos del DOM:

```
it ('Descripción de la prueba',()=>{
        //Inicialización de los datos 'bindeados'
        objeto.detectChanges();//Actualiza el DOM
        const elementos = debEle.queryAll(By.css('.nombre_clase'));
        expect(elementos).toBeTruthy('No se han obtenido elementos);
        expect(elementos.length).toBe(número_de_instancias, "Deberían ser n elementos");
}
```

queryAll vs query

ANGULAR

- Testing
  - Pruebas sobre componentes:
    - A partir de la referencia a los elementos (let debEle:DebugElement) se puede obtener referencias a los elementos contenidos en el DOM:
      - const *elemento* = debEle.query(By.css('.clase:first-child'));
      - expect(*elemento*).toBeTruthy('El elemento debe estar creado');
      - const *subelemento* = *elemento*.query(By.css('……..'));
      - expect(*subelemento*.nativeElement.textContent).toBe("cadena", 'El contenido debe ser …");

ANGULAR

- Testing
  - Método spyOn
    - Permite "espiar" el uso de un determinado objeto.
      - spyOn(*nombreObjeto,'nombreMétodo'*);
    - Para evaluar el resultado se debe referenciar al objeto y al método "espiado" para determinar cuantas veces se ha ejecutado:
      - expect(*nombreObjeto.nombreMétodo*).toHaveBeenXXX
    - Se pueden utilizar objetos "ficticios" para evaluar su uso y no generar instancias innecesarias en las pruebas.
      - const *objeto* = jasmine.createSpyObj('*nombre-clase*', ['*nombreMétodo1','nombreMétodo2']);